

RUTHLESS SaaS Security Architecture - FINAL v2.0

The Un-Hackable Blueprint for Integration-Heavy SaaS with Payments + AI

Combines 70+ pages from all your guides. Zero Trust. Defense-in-Depth. Fail-Closed. Assume Breach.

0. How You Get Hacked (Why This Architecture Exists)

Traditional SaaS Top Attack Vectors (2025-2026 Data)

1. Stolen/abused valid credentials (credential stuffing, password spray, reuse of breached passwords)
2. Phishing & social engineering (AiTM phishing stealing session cookies bypassing MFA, vishing, help-desk)
3. Misconfigurations (overly permissive sharing, public S3/DB, missing MFA enforcement)
4. OAuth / token abuse & third-party integration compromise
5. Broken access control (BOLA/IDOR, missing tenant isolation, privilege escalation)
6. Session / token hijacking, MFA fatigue/bypass
7. Ghost/dormant accounts, guest/external collaborator abuse, service account compromise
8. API abuse / no rate limiting, Injection (SQL/NoSQL/XSS/Command), Malicious dependencies, Compromised CI/CD, Insider threats, Data exfiltration post-login

AI SaaS Extra Attack Vectors (OWASP LLM Top 10)

1. Prompt Injection #1 (direct + indirect / zero-click / stored / cross-agent)

- 2. Sensitive Information Disclosure (system prompts, other tenants data, secrets, RAG boundary failures)
- 3. Excessive Agency / Tool Misuse (agents performing unintended actions)
- 4. Cross-tenant data leakage in multi-tenant RAG systems
- 5. LLM Jacking / API key abuse, Model Denial of Service (unbounded consumption / Denial of Wallet)
- 6. Insecure output handling, Training data / model poisoning, Supply chain attacks on models/datasets/frames
- 7. Hidden context / system prompt leakage, Jailbreaking / safety filter bypass, Model inversion or extraction
- 8. Invisible reasoning-trace injection, Tool/plugin abuse leading to RCE or exfiltration, Stolen LLM provider credentials
- 9. Over-permissioned service accounts used by AI agents

RUTHLESS Rule: If one of these can happen, the architecture must stop it at 3 independent layers.

1. Core Design Principles (Non-Negotiable)

Principle	Ruthless Implementation
Zero Trust	Verify every request - user, service, integration, webhook, file, AI output. No trust by network location.
Least Privilege	Users, services, integrations, AI tools, workers, DB roles get minimal scopes. JIT elevation only.
Defense in Depth	Edge -> Gateway -> Identity -> AuthZ -> App -> Integration Proxy -> Data. One failure != total compromise.
Tenant Isolation is Security Boundary	Every query, cache key, queue message, file, search result, AI context, audit event carries <code>tenant_id</code> . Enforced at DB level.

Principle	Ruthless Implementation
Secure by Default & Fail Closed	New tenants safest config. If authZ, tenant resolution, signature verification, policy evaluation fails -> DENY.
Assume Breach	Design so one compromised account/integration/service/tenant cannot compromise whole platform. Micro-segmentation, blast radius limit.
Data Minimization	Don't collect what you don't need. Tokenize what you must keep.
Immutability	Infra as Code, append-only audit logs (WORM), no manual prod changes.

2. Reference Architecture (13 Layers)

```
1  INTERNET (Untrusted: Users, Attackers, Third-Party Apps, AI Models, F
1  |
1  [ L1 Edge: CDN / WAF / DDoS / Bot Management / IP Reputation ]
1  |
1  [ L2 API Gateway / Identity Proxy: AuthN + AuthZ + Rate Limit + Schem
1  |
1  +-----+
1  | L3 Identity Platform: Auth0/Clerk, MFA/Passkeys, SSO/SAML/OIDC, SCI
1  | L4 Authorization Platform: RBAC / ABAC / ReBAC, Policy Engine, Risk
1  +-----+
1  |
1  +-----+
1  | Applications      | Integration Platform  | AI Platform |
1  | CRM/Tasks/Billing | OAuth Connectors      | AI Gateway  |
1  | Automation        | Webhooks (In/Out)      | RAG (per-tenant) |
1  | Analytics          | Sync / Import/Export    | Agents / Tools / Model
1  +-----+
1  |
1  +-----+
1  | L Security Services: Secrets Vault + KMS + DLP + Fraud/Abuse + Audi
1  +-----+
```

```

1 |
1 [ L5 Database (Postgres RLS) + Storage (S3 SSE-KMS per-tenant) + Cach
1 [ L6 Queues / Event Bus: Secure async jobs, tenant-aware workers, iso
1 [ L7 Observability: SIEM / Logs / Metrics / Trace / Detection / Alert
1 [ L8 Incident Response / Backup / DR / Recovery ]

```

Trust Zones: Zone 0 Internet -> Zone 1 DMZ (Edge) -> Zone 2 App Zone -> Zone 3 Data Zone (most trusted). Traffic only goes down. Every crossing logs.

3. Layer-by-Layer RUTHLESS Controls

L1 Edge & Network Security

- CDN: Cloudflare (recommended) / AWS CloudFront + Shield Advanced / Fastly
- WAF: OWASP Core Rule Set, block SQLi/XSS/RCE patterns at edge
- Bot Management: Cloudflare Bot Management / AWS Bot Control - block credential stuffing scanners
- Rate Limit at Edge: 100 req/s per IP, 10 req/s for /auth/*, challenge page after threshold
- DDoS: Anycast, auto-absorb L3/L4/L7
- IP Reputation: Block Tor, known malicious ranges
- TLS: TLS 1.3 only (min 1.2), HSTS preload, no mixed content

L2 Identity & Authentication (IdP) - Never Build Your Own

- Provider: Auth0 / Clerk / Keycloak / Supabase Auth + custom MFA
- Passwords: Argon2id preferred (bcrypt/scrypt fallback), unique salts, 12+ chars server-enforced, HaveIBeenPwned API check, never log passwords, never plaintext
- MFA: Enforced for all users especially admins, phishing-resistant WebAuthn / FIDO2 / Passkeys first-class, TOTP secondary, avoid SMS, recovery codes + secure account recovery
- Passwordless: Passkeys as primary option

- SSO: SAML 2.0 / OIDC for enterprise tenants
- SCIM 2.0: Automated provisioning/deprovisioning
- Sessions: Short-lived access tokens 15-30 min + refresh token rotation + revocation, secure cookies HttpOnly Secure SameSite=Strict, idle timeout (30 min) + absolute timeout (12h), server-side invalidation on logout + security changes, device/session inventory, logout-all-devices, suspicious-session detection (impossible travel)
- Hardening: Rate limit + exponential backoff + account lockout on auth endpoints, progressive delays, credential stuffing detection, consistent responses that don't reveal account existence, anomaly detection on login patterns

L3 Authorization & Tenant Isolation (CRITICAL - 60% of SaaS leaks)

- RBAC + ABAC/ReBAC where needed, least privilege everywhere, fine-grained per-resource permissions
- AuthZ checked on EVERY API request (backend, not frontend). Frontend hides button != security.
- Re-authentication for high-risk actions (delete, export, billing change)
- Regular access reviews for privileged accounts, Just-in-Time (JIT) privilege elevation
- Service-to-service auth via mTLS or short-lived workload identities
- **Multi-Tenancy Enforcement:** Design isolation from day 1. Database-level enforcement (PostgreSQL Row-Level Security) - application code never solely responsible.

```
sql
```

```
1
```

```
CREATE POLICY tenant_isolation ON all_tables FOR ALL
USING (tenant_id = current_setting('app.current_tenant_id')::uuid);
```

```

1 - Isolation Models:
1 - Shared DB + RLS: Strong if done correctly, Lowest cost, Best for Mo
1 - Schema-per-tenant: Stronger, Medium cost, Higher compliance
1 - DB-per-tenant: Strongest, Highest cost, Regulated/enterprise
1 - Additional: Separate or hierarchical encryption keys per tenant, te
1
1 ### L4 Application Security
1 - Input Validation: Server-side allow-list (Zod/Pydantic), reject unk
1 - Output Encoding: Escape all rendering contexts
1 - Injection Protection: Parameterized queries only, ORM safe mode
1 - File Uploads: Scan with ClamAV, validate MIME not extension, store
1 - CSRF: Double-submit token + SameSite, state-changing operations onl
1 - Security Headers: CSP with nonce, X-Frame-Options DENY, X-Content-T
1 - Error Handling: Generic errors to user, details only in server logs
1
1 ### L5 API Security
1 - API Gateway as single entry point (Kong / AWS API Gateway / Traefik
1 - Scoped API Keys: Per-tenant, per-integration, least privilege, shor
1 - Signing: HMAC SHA-256 for webhook verification, timestamp validatio
1 - Throttling: Per tenant + per endpoint + per API key, sliding window
1 - Idempotency: Idempotency-Key header required for billing, state cha
1 - OWASP API Top 10: Especially BOLA (Broken Object Level Authorizatio
1 - Versioning & Deprecation: Secure patching path
1
1 ### L6 Third-Party Integration Security (Highest Risk)
1 **Outbound (Your SaaS -> External Apps):**
1 - Use OAuth 2.0 / OIDC with least privileged scopes possible
1 - Short-lived access tokens + refresh token rotation, token revocatio
1 - Store tokens encrypted in dedicated secrets manager (HashiCorp Vault
1 - Encrypt tokens at rest with tenant-specific keys, support BYOK
1 - Implement circuit breakers and rate limiting on outbound calls, tok
1 - Integration Gateway / Proxy Pattern: App -> Proxy (with tenant_id)
1
1 **Inbound (External Apps -> Your SaaS):**
1 - Dedicated API keys or OAuth clients per integration, strict scope l
1 - Webhook endpoints must validate signatures and be rate-limited, nev
1 - All webhook payloads treated as untrusted
1
1 **Data Import / Export Security:**
1 - All import/export jobs run under least privileged service identity
1 - Temporary signed URLs for large file transfers (short expiry 10 min
1 - Data transformation pipelines must respect tenant boundaries, full
1 - Support customer-managed encryption keys (BYOK) for enterprise
1 - Import Pipeline: `Upload -> S3 Quarantine (private) -> SQS -> Isola
1 - Export: Async job only, never direct DB dump, generate signed URL,
1
1 ### L7 Payment System Security (PCI DSS Scope Reduction)

```

```

1 - Never store full card data in your systems - Golden Rule
1 - Use PCI DSS Level 1 compliant provider (Stripe, Adyen, Braintree)
1 - Tokenization only - your system only ever sees tokens (tok_XXX, pi_
1 - Isolate payment services in a separate network segment, strict access
1 - Client-side tokenization (Stripe.js / PayPal SDK), application server
1 - Webhooks from payment provider must be signature-verified (Stripe-S
1 - Protect billing APIs with extra rate limiting + ownership checks
1
1 ### L8 Data Security & Encryption
1 | State | Ruthless Control |
1 |---|---|
1 | In Transit | TLS 1.3 everywhere (including internal service-to-service)
1 | At Rest | AES-256, database TDE + field-level / application-level encryption
1 | In Use | Consider confidential computing / enclave for highest sensitivity
1 | Keys | Centralized KMS (AWS KMS / GCP KMS / Azure Key Vault / Vault)
1 | Classification | Public / Internal / Confidential / Restricted, lab
1 | DLP | Data Loss Prevention controls, block mass export of Restricted
1 | Logging | Never log secrets or sensitive PII; mask data in non-production
1 | Backups | Encrypted, immutable (WORM), tested regularly (monthly restore)
1 | Retention | Clear retention and deletion policies, GDPR/CCPA/DPDP requirements
1
1 ### L9 Infrastructure & Platform Security
1 - Cloud IAM: Least-privilege, no long-lived access keys, use workload identities
1 - Private networking: No public databases, VPC with private subnets,
1 - Secrets: Managed via dedicated secrets manager (Vault / Secrets Manager)
1 - IaC: Terraform / CloudFormation with security scanning (Checkov, tfsec)
1 - Containers: Image scanning + least-privilege runtime (non-root, read-only)
1 - K8s: Pod Security Standards, network policies, secrets encryption at rest
1 - Hardening: CIS benchmarks for OS, DB
1
1 ### L10 AI/LLM Security (If Applicable) - The New Frontier
1 - AI Gateway: Single chokepoint for all LLM calls, logs prompts/responses
1 - Prompt Injection Defense: Treat all external content (docs, webpage) as untrusted
1 - RAG Security: Per-tenant vector DB namespace (Pinecone/Qdrant namespaces)
1 - Agent Safety: Tool allow-list, approval required for destructive tool calls
1 - Resource Quotas: Strict token limits, context window flood protection
1 - Prompt Shielding: System prompt hardening, output handling (never return secrets)
1 - Model Security: Supply chain verification for models/datasets, training data provenance
1 - Secrets: Never put secrets in prompts, LLM provider API keys in Vault
1
1 ### L11 Monitoring, Logging & Incident Response
1 - Comprehensive audit logging: Authentication, authorization failures
1 - Structured, immutable logs with long retention (1 year min), WORM storage
1 - Detection: Impossible travel, mass export (1000+ records in 5 min),
1 - Documented and tested Incident Response plan, regular tabletop exercises
1 - Metrics: Failed auth rate, token vault access, export volume, integration health
1
1 ### L12 Secure Development Lifecycle (SDLC)
1 - Threat modeling for new features and integrations (STRIDE)
1 - Peer code review mandatory for auth, payment, tenant isolation code
1 - Automated security scanning in CI/CD: SAST (Semgrep, SonarQube), DAST

```

```

1 - Regular independent penetration testing (annual + after major chang
1 - Security ownership within engineering team, security champions prog
1 - Dependency update policy (Dependabot auto-merge for patches)
1
1 ### L13 Compliance & Governance
1 - Frameworks: SOC 2 Type II (B2B enterprise requirement), ISO 27001,
1 - Start preparing evidence early: Policies, logs, access reviews, inc
1 - Data Processing Agreements with sub-processors, privacy policy, ToS
1
1 ### L14 Disaster Recovery & Business Continuity
1 - RTO: Payments 1h, Core app 4h, Analytics 24h
1 - RPO: Financial 0 min (sync replication), User data 5 min (PITR), Ap
1 - Architecture: Primary region active, secondary warm standby, DB Mul
1
1 ---
1
1 ## 4. Secrets & Configuration Management
1 - No secrets in code, repo, env files, logs, frontend
1 - Vault: HashiCorp Vault / AWS Secrets Manager / Doppler / Infisical
1 - Rotation: Automated 90 days, on employee exit, on suspected breach
1 - Access: Audit who accessed which secret when
1
1 ---
1
1 ## 5. Technology Stack Reference (Battle-Tested)
1
1 | Layer | Recommended | Notes |
1 |---|---|---|
1 | CDN / WAF / Edge | Cloudflare | WAF + DDoS + Bot management + best
1 | API Gateway | Kong / AWS API Gateway / Traefik | Single entry, sche
1 | Identity | Auth0 / Clerk / Keycloak / Supabase Auth | Don't build a
1 | SSO Enterprise | WorkOS / Auth0 | SAML + OIDC |
1 | DB | PostgreSQL with RLS + pgcrypto | Database-enforced isolation |
1 | Secrets | HashiCorp Vault / AWS Secrets Manager / Doppler | Per-ten
1 | KMS | AWS KMS / GCP KMS / Azure Key Vault | Envelope encryption, BY
1 | Object Storage | S3 / GCS with strict IAM + SSE-KMS + private | No
1 | Queue | SQS / BullMQ / Cloud Tasks | Tenant-aware workers |
1 | SAST | Semgrep / SonarQube | Run on every PR |
1 | SCA / Dependencies | Snyk / Dependabot / Trivy | Daily scans |
1 | Secret Scanning | GitGuardian / detect-secrets | Pre-commit + CI |
1 | IaC Scanning | Checkov / tfsec | Terraform security |
1 | SIEM / Monitoring | Datadog Security / Elastic / Grafana + Loki | R
1 | Pen Testing | Cobalt / Synack | Annual |
1 | Bug Bounty | HackerOne / Bugcrowd | After launch |
1 | Payments | Stripe | PCI DSS Level 1, best DX |
1 | Compliance Automation | Vanta / Drata / Secureframe | SOC2 evidence
1
1 ---
1
1 ## 6. Implementation Roadmap (Ruthless Order)
1

```

```

1  **Phase 1 - Foundation (Month 1-2) Critical - No Users Without This:**
1  Multi-tenancy design with database-enforced isolation + Strong identi
1
1  **Phase 2 - Core App Security (Month 3-4):**
1  RBAC / fine-grained authorization + API security & input validation +
1
1  **Phase 3 - Integrations (Month 5-6):**
1  Secure OAuth connector framework + Token encryption & lifecycle manag
1
1  **Phase 4 - Payments (Month 6-7):**
1  Tokenized payment flow + Network isolation of payment services + Webh
1
1  **Phase 5 - Advanced & Maturity (Month 8-12):**
1  Anomaly detection + Customer-managed keys (BYOK) + Formal compliance
1
1  ---
1
1  ## 7. Pre-Launch Ruthless Checklist
1
1  **Must-have before first real user:**
1  - [ ] Argon2id password hashing, 12+ chars, HaveIBeenPwned check, MFA
1  - [ ] TLS 1.2+ everywhere, HSTS, security headers on all responses
1  - [ ] Input validation all endpoints, parameterized queries, CSRF pro
1  - [ ] Multi-tenant isolation enforced (tenant_id on all queries + RLS
1  - [ ] No secrets in code, secrets in manager, DB not public, security
1  - [ ] Auth events logged, error alerting, uptime monitoring
1  - [ ] Using PCI-compliant payment provider, no card data touches your
1  - [ ] Privacy policy, ToS, DPA with sub-processors
1
1  **Must-have before enterprise customers:**
1  - [ ] SOC 2 Type II in progress, pen test within 12 months, SAML SSO,
1  - [ ] Audit log available to customers, data export + deletion (GDPR)
1  - [ ] Incident response SLAs defined, bug bounty / responsible disclo
1  - [ ] Per-tenant KMS keys, PII Vault tokenization, status page, DLP,
1
1  ---
1
1  ## 8. Final Architecture Diagram (Text)

```

External Users / Admins / Third-Party Apps / AI Models

|

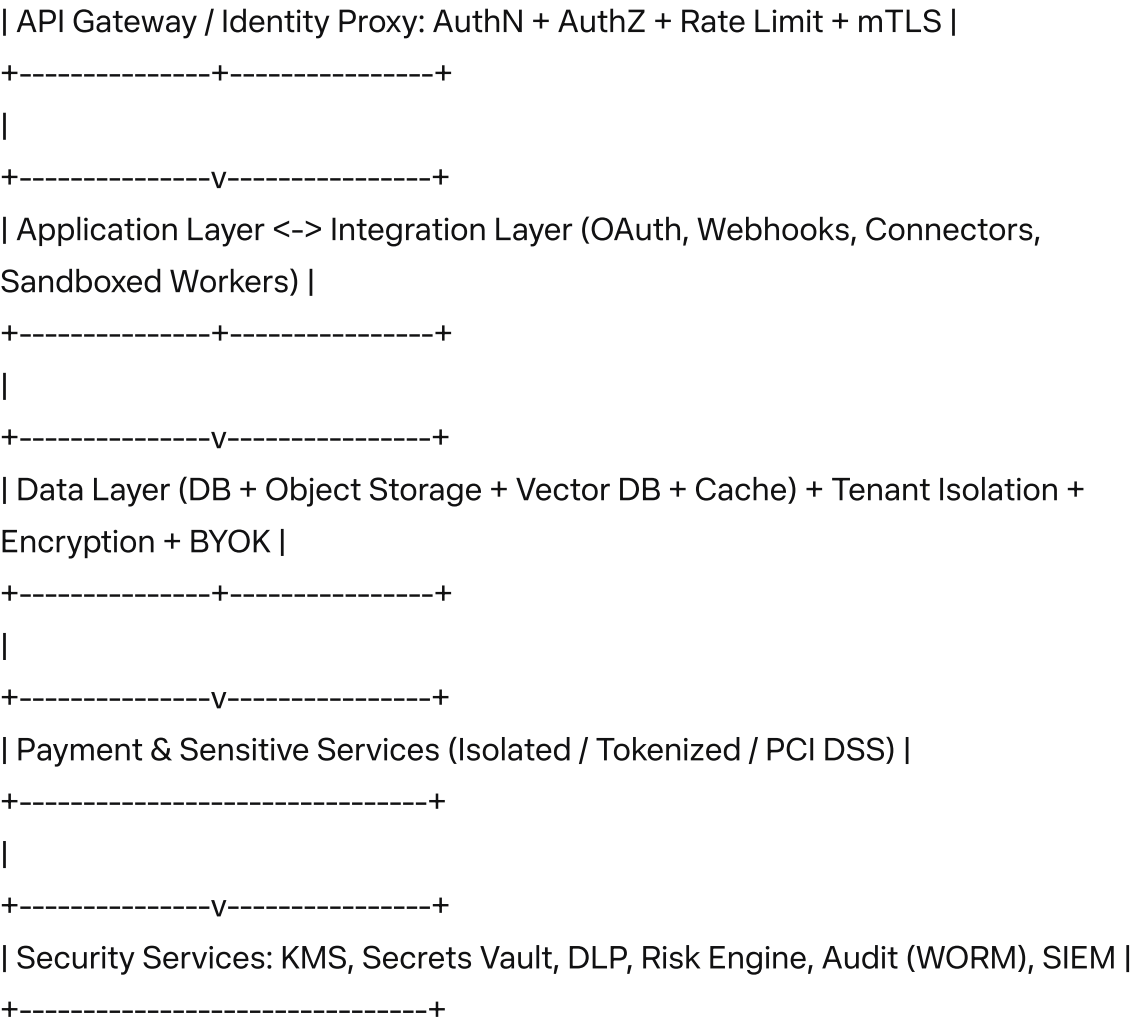
+-----V-----+

| Edge Layer: WAF + CDN + DDoS + Bot Protection + Rate Limit |

+-----+-----+

|

+-----V-----+



1
1 Each layer independently secured. Failure of one layer should not com
1
1 ---
1
1 *Generated: August 2026 | Security First Edition | Version 2.0 RUTHLE
1 *Use as living reference. Review quarterly and after any incident.*
1 *Disclaimer: Consult qualified security professional for your specifi